

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Web Programlama

FastAPI I: HTTP'yi Koda Dökmek

Hafta 6

FastAPI I: HTTP'yi Koda Dökmek

Doç. Dr. Tolga Berber

KTÜ Fen Fakültesi, Bilgisayar Bilimleri Bölümü · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim Neden Önemli?

Geçen hafta oluşturduğunuz `Bilet[]` yalnızca `App`'in yerel belleğinde (state) yaşıyordu.

- Sayfayı yenilediğinizde bu biletler kayboluyordu — veri hiç bilgisayarınızın dışına çıkmadı
- Bir arkadaşınız aynı siteyi başka bir sekmede açtığında bu biletleri hiç göremiyordu — "aynı etkinliği iki kullanıcı aynı anda görür mü?" sorusunun cevabı henüz Hayırdı
- Bu hafta ilk kez gerçek bir sunucu yazıyorsunuz — Etkinlik/Koltuk/Bilet için bellekte çalışan bir CRUD API'si; veri artık tek, paylaşılan bir süreçte yaşayacak

Hf.1'in `curl` komutları bugün ilk kez kendi yazdığınız kodun ürettiği gerçek bir cevaba gidecek.

Bu Hafta

◎ Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

1. venv kurup `uvicorn` ile ilk FastAPI uygulamanızı çalıştırıp yol/sorgu/gövde parametrelerini tanımlamak
2. `BaseModel`+`Field` kısıtlarıyla giriş/çıkış modelleri tanımlayıp camelCase/snake_case köprüsünü kurmak
3. Bellekte CRUD uç noktaları yazıp `HTTPException` ve `409` iş kuralını uygulamak
4. `/docs`, `Depends` ve `TestClient` ile API'nizi belgeleyip test etmek
5. Bunları birleştirip Etkinlik/Bilet Sistemine ilk sunucu katmanını eklemek

BÖLÜM 1

Ünite 1 — İlk Uygulama, Yol/Sorgu/Gövde Parametreleri

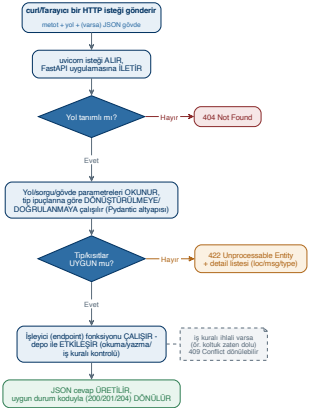
2 • İçerik venv, uvicorn, Yol/Sorgu Parametreleri

- Sanal ortam (venv): `python -m venv .venv` ile projeye özel, izole bir Python kurulumu; farklı projelerin farklı paket sürümlerine ihtiyacı olabileceğinden küresel kuruluma tercih edilir
- FastAPI bir ÇERÇEVEdir — hangi yola hangi fonksiyonun cevap vereceğini tanımlar; TCP'yi dinleyip HTTP'yi konuşan gerçek süreç `uvicorn`dur (bir ASGI sunucusu)
- Yol parametresi: `@app.get("/etkinlikler/{etkinlik_id}")` — `etkinlik_id: int` tip ipucuyla URL'deki metin otomatik `int`'e çevrilir
- Sorgu parametresi: yol şablonunda geçmeyen bir parametre (`mekan: str | None = None`) otomatik `?mekan=...` biçiminde isteğe bağlı sayılır

Tanım: Yol parametresi (*path parameter*)

URL yolunun süslü parantezle işaretli, değişken bir parçasını yakalayan parametredir.

İstek Yaşam Döngüsü: `curl`'dan Cevaba



Analoji: Malzeme Çekmecesi ve Gişe Binası

Analoji

Sanal ortamı, bir mutfakta birden fazla tarif (proje) hazırlarken her tarifin kendi malzeme setini (paket sürümlerini) karıştırmadan ayrı bir çekmecede tutmaya benzetin: bir tarifin malzemesi diğeriyle asla yer değiştirmez.

Analoji

`uvicorn`/FastAPI ilişkisini bir gişe binası ile gişe memurunun kuralına benzetin: bina (`uvicorn`) kapıyı açar, geleni içeri alır; hangi pencereye yönlendirileceğini, pencerede ne sorulacağını belirleyen kural kitapçığı ise FastAPI'nin yazdığınız kodudur.

Kod: FastAPI Nesnesi ve İlk Uç Nokta — `main.py`

```python
app = FastAPI(
    title="KTÜ Etkinlik ve Bilet Sistemi API",
    version="0.1.0",
)
```

```python
@app.get("/")
def kok_rota() -> dict[str, str]:
    return {"mesaj": "KTÜ Etkinlik ve Bilet Sistemi API'sine hoş
        geldiniz"}
```

`uvicorn app.main:app --reload` çalışırken `curl -i http://127.0.0.1:8000/` bu JSON'u döndürür.

BÖLÜM 2

Ünite 2 — Pydantic Modelleri ve Doğrulama

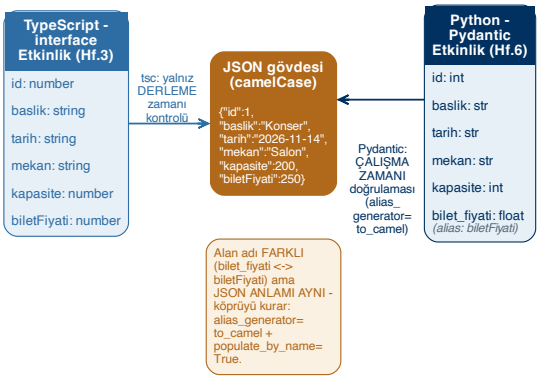
2 • İçerik Doğrulama, Giriş/Çıkış Modeli, camelCase Köprüsü

- `BaseModel`: tip ipuçlarından otomatik doğrulama+ayrıştırma kazanan Python sınıfı — Hf.3'ün `interface`'i yalnızca derleme zamanı sözleşmesiyken, Pydantic çalışma zamanında da veriyi denetler
- `Field` kısıtları: `kapasite: int = Field(gt=0)` — yalnız tip değil değer aralığı da doğrulanır
- Giriş/çıkış modeli ayrımı: `EtkinlikOlustur` (henüz `id`'si yok) vs `Etkinlik` (`id` atanmış tam nesne)
- camelCase/snake_case köprüsü: ortak `KamelModel` taban sınıfı, `alias_generator=to_camel` ile Hf.3'ün TS arayüzleriyle birebir aynı JSON sözleşmesini kurar

Tanım: Takma ad (*alias*)

Bir modelin dış dünyaya (JSON) farklı bir adla göründüğü alan eşlemesidir.

TS Arayüzü ↔ Pydantic Modeli Eşlemesi



Analoji: Gişe Memurunun Kontrolü ve İki Kurumda Aynı Ad

Analoji

Pydantic doğrulamasını bir gişe memurunun, verdiğiniz formu imzalamadan önce her alanı kontrol etmesine benzetin: kapasiteyi negatif girmişseniz memur formu kabul etmez, hangi alanın yanlış olduğunu söyler — form işleme hiç girmez.

Analoji

camelCase/snake_case köprüsünü aynı kişinin iki farklı kurumda farklı biçimde yazılan adına benzetin: biçim farklıdır, anlam aynıdır — bir eşleme tablosu (alias) ikisini birbirine bağlar.

Kod: camelCase Köprüsü — `KamelModel`

```python
class KamelModel(BaseModel):
    """snake_case Python alanlarını camelCase JSON'a bağlayan taban
       sınıf."""

    model_config = ConfigDict(alias_generator=to_camel,
       populate_by_name=True)
```

`alias_generator=to_camel` her `snake_case` alan için otomatik bir `camelCase` takma ad üretir; `populate_by_name=True` modelin hem takma adla hem Python alan adıyla doldurulabilmesini sağlar.

Kod: Giriş/Çıkış Modeli Ayrımı — `Etkinlik`

```python
class EtkinlikOlustur(KamelModel):
    """İstemcinin gönderdiği, henüz id'si olmayan etkinlik verisi."""

    baslik: str = Field(min_length=1)
    tarih: str
    mekan: str = Field(min_length=1)
    kapasite: int = Field(gt=0)
    bilet_fiyati: float = Field(gt=0)

class Etkinlik(EtkinlikOlustur):
    """Sunucunun döndürdüğü, kimliği atanmış tam etkinlik nesnesi."""

    id: int
```

`EtkinlikOlustur`'dan türeyen `Etkinlik` yalnız `id` ekler — Hf.1'in sunum/iş/veri katmanı ayrımının somut bir yansımasıdır.

422 Doğrulama Hatası: Anatomisi

`POST /etkinlikler` gövdesinde `kapasite: -5` gönderildiğinde:

Beklenen Çıktı

```
HTTP/1.1 422 Unprocessable Entity
{
  "detail": [
    {"loc": ["body", "kapasite"],
     "msg": "Input should be greater than 0",
     "type": "greater_than"}
  ]
}
```

`loc` hangi alanı, `msg`/`type` açıklama ve türü gösterir — doğrulama başarısız olunca işleyici fonksiyon hiç çağrılmaz, nesne depoya asla yazılmaz.

BÖLÜM 3

Ünite 3 — CRUD, Durum Kodları, Hata Cevapları

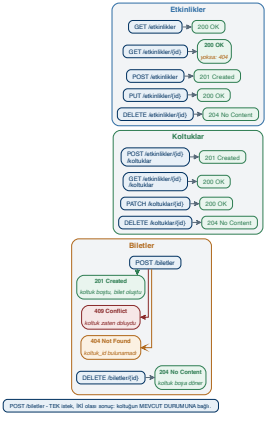
2 • İçerik CRUD, Durum Kodları, Koltuk Rezervasyonu

- Bellekte depo: `dict[int, Etkinlik]` — süreç kapandığında (`--reload` dahil) veri tamamen kaybolur
- `POST`→`201`, `GET`→`200`, `PUT`/ `PATCH`→`200`, `DELETE`→`204` — doğru kullanım sıradaki matriste
- `HTTPException(404)`: istenen `id` depoda yoksa
- Koltuk rezervasyonu iş kuralı: koltuk `'dolu'` ise `POST /biletler` `409 Conflict` ile reddedilir — istek format olarak geçerlidir, sorun mevcut durumla çelişmesidir

Tanım: Çakışma (*conflict*) hatası

İsteğin biçimi geçerli ama sunucudaki mevcut durumla çeliştiği için reddedildiğini bildiren `409` durumudur.

CRUD Durum Kodları Referans Matrisi



Analoji: İki Kişi Aynı Koltukta ve Kat Kat Ayrılan AVM

Analoji

`409 Conflict`'i aynı numaralı otobüs koltuğunu iki kişinin ayrı gişelerden almaya çalışmasına benzetin: ikinci biletin biçimi tamamen doğru olabilir, ama gişe memuru "bu koltuk zaten satıldı" diyerek ikinci satışı reddeder — sorun formatta değil, mevcut durumdadır.

Analoji

`APIRouter` ile ayrıştırmayı bir alışveriş merkezindeki her katın kendi tabelasını/görevlisini taşımasına benzetin: "Etkinlikler katı", "Koltuklar katı", "Biletler katı" diye ayırmak, hangi işin nerede yapıldığını hemen belli eder.

Kod: Koltuk Rezervasyonu İş Kuralı — `bilet_olustur`

```python
@router.post("", response_model=Bilet, status_code=201)
def bilet_olustur(veri: BiletOlustur, depo: Depo = Depends(get_depo)) -> Bilet:
    koltuk = depo.koltuk_getir(veri.koltuk_id)
    if koltuk is None:
        raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
    if koltuk.durum == "dolu":
        raise HTTPException(status_code=409, detail="Bu koltuk zaten satılmış")
    bilet = depo.bilet_ekle(veri)
    depo.koltuk_guncelle(koltuk.id, "dolu")
    return bilet
```

Koltuk yoksa `404`, `'dolu'` ise `409` — aksi halde bilet oluşturulur ve koltuğun `durum`'u `'dolu'`ya çevrilir.

Kod: Bulunamama Hatası — `etkinlik_detay`

```python
@router.get("/{etkinlik_id}", response_model=Etkinlik)
def etkinlik_detay(etkinlik_id: int, depo: Depo = Depends(get_depo))
    -> Etkinlik:
   etkinlik = depo.etkinlik_getir(etkinlik_id)
   if etkinlik is None:
      raise HTTPException(status_code=404, detail="Etkinlik
         bulunamadı")
   return etkinlik
```

Bu desen her rota dosyasında (Koltuklar, Biletler) tekrarlanır — `HTTPException(404)` Python'ın `try/except`'ine benzer ama doğrudan bir HTTP durum koduna bağlıdır.

BÖLÜM 4

Ünite 4 — OpenAPI Belgeleri ve Bağımlılık Enjeksiyonu

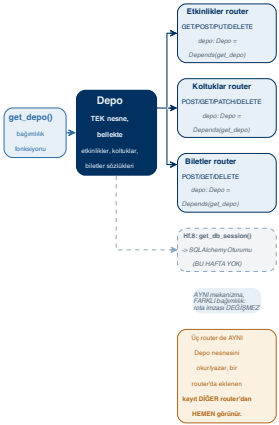
2 • İçerik /docs, Depends, TestClient

- `/docs` (Swagger UI): FastAPI'nin yazdığınız her rotadan otomatik ürettiği, tarayıcıda deneyebileceğiniz interaktif belge — hiçbir ek kod yazmadan
- `/openapi.json`: aynı belgenin makine-okunabilir JSON şeması — Hf.3'ün TS `interface`'leriyle birebir örtüşür
- `Depends`: bir rota fonksiyonunun ihtiyaç duyduğu nesneyi (depo) kendisi oluşturmak yerine `depo: Depo = Depends(get_depo)` ile FastAPI'den istemesi
- `TestClient`: gerçek bir sunucu ayağa kaldırmadan, `pytest`'in `assert` disipliniyle ilk API testlerinizi yazmak

Tanım: Bağımlılık enjeksiyonu (*dependency injection*)

Bir fonksiyonun ihtiyaç duyduğu nesneyi kendisi oluşturmak yerine dışarıdan (çerçeveden) almasıdır.

Depends: Tek Depo, Üç Router



Analoji: Kendiliğinden Güncellenen Menü ve Hazır Defter

Analoji

`/docs`'u bir restoranın mutfak tarafından otomatik güncellenen menüsüne benzetin: yeni bir yemek (rota) eklediğinizde menü kendiliğinden güncellenir, garsonun elle bir kağıda yeni yemeği yazmasına gerek kalmaz.

Analoji

`Depends`'i bir resepsiyon görevlisine, ihtiyaç duyduğu kayıt defterini kendisi satın almak yerine binadan hazır verilmesine benzetin: görevli defterin nereden geldiğiyle ilgilenmez, yalnızca "bana bir defter lazım" der.

Kod: Bağımlılık Fonksiyonu — get_depo

```python
def get_depo() -> Depo:
    """Rota fonksiyonlarına PAYLAŞILAN tek Depo nesnesini enjekte eder
        .

    Hf.8'de bu fonksiyonun yerini bir veritabanı oturumu enjekte eden
    get_db_session() alacak; rota fonksiyonlarının bağımlılık ALMA
    biçimi değişmeyecek, yalnızca bağımlılığın NE OLDUĞU değişecek.
    """
    return depo
```

tüm rotalar (Etkinlikler/Koltuklar/Biletler) aynı, tek `depo` nesnesini kullanır — bu sayede bir `POST /etkinlikler` hemen ardından bir `GET /etkinlikler`'de görünür.

Kod: Testte Temiz Depo — `TestClient`

```python
@pytest.fixture
def client():
    test_depo = Depo()
    app.dependency_overrides[get_depo] = lambda: test_depo
    with TestClient(app) as test_client:
        yield test_client
    app.dependency_overrides.clear()
```

`app.dependency_overrides[get_depo]` ile her teste temiz, boş bir depo verilir — bu `Depends` olmasaydı mümkün olmazdı, testler arasında veri sızıntısı oluşurdu.

BÖLÜM 5

Özet

Özet

Özet

- venv + `uvicorn` + yol/sorgu parametresi: ilk çalışan FastAPI uç noktanızı yazdınız
- Pydantic `BaseModel`/`Field` + giriş/çıkış modeli + camelCase köprüsü: Hf.3'ün TS'iyle örtüşen modeller kurdunuz
- Bellekte CRUD + durum kodları + `HTTPException` + koltuk rezervasyonu `409`: Etkinlik/Koltuk/Bilet uç noktalarını yazdınız
- `/docs` + `Depends` + `TestClient`: API'nizi kendi kendini belgeler ve test edilebilir hâle getirdiniz

5 • İzleme Aktarım — Başka Nerede?

- Bugünkü venv+pip+`uvicorn` iş akışı, dönem boyunca her Python sunucu projesinde (Hf.8, Hf.10, Hf.13) aynen tekrarlanacak
- `KamelModel`/giriş-çıkış modeli deseni Hf.8'de SQLAlchemy modelleriyle, Hf.10'un kullanıcı kayıt formunda (şifre alanı girişte var, çıkışta asla dönülmez) aynı mantıkla sürecek
- `HTTPException`/durum kodu disiplini Hf.10'un `401`'i, Hf.11'in `403`'ünde aynı mekanizmayla; koltuk rezervasyonu iş kuralı Hf.8'de "aynı anda iki isteğin yarışması" sorunuyla yeniden karşınıza çıkacak
- `Depends` deseni Hf.8'de `get_db_session`'a, Hf.10'da `get_mevcut_kullanici`'ya genişleyecek
- Bu haftanın sunucusu hâlâ tarayıcıya bağlı değil — Hf.7'de `fetch` ile bağlanmaya çalıştığınızda bir engelle (CORS) karşılaşacaksınız, çözümü Hf.7'dedir

Sırada Ne Var?

Bu hafta lab'da

`app/modeller.py`, `app/depo.py` ve `app/rotalar/` altındaki router'ları kendi elinizle yazıp `uvicorn app.main:app --reload` ile çalıştıracak, `python -m pytest -q` ile testlerinizi geçirecek ve Etkinlik/Bilet Sisteminin ilk sunucu katmanını kuracaksınız.

- Kendinizi Sınayın her üniteyi kapsıyor: `/abc` yoluna neden `404` değil `422` döner, negatif `kapasite` gönderilince `detail` ne içerir, ikinci rezervasyon denemesi neden `400` değil `409` döner, `Depends(get_depo)` olmasaydı ne değişirdi
- Gelecek hafta: React III — API Tüketimi ve Yönlendirme — bu haftanın sunucusuna `fetch` ile bağlanacaksınız